

Exp No:1	SIMULATION OF CLOUD SCHEDULING USING CUSTOM BROKER IN CLOUDSIM
Date:	

AIM:

To simulate a cloud computing environment using the CloudSim toolkit and implement a custom scheduling algorithm that is not available in the default CloudSim scheduling policies.

PROCEDURE:

Step 1: Install Java and Eclipse

- Install JDK 1.8
- Install Eclipse IDE for Java Developers
- Download CloudSim 3.0.3
 - <https://github.com/cloudslab/cloudsim/releases/tag/cloudsim-3.0.3>
 - On that page you will find the zip package containing: cloudsim-3.0.3.zip
 - Right-click on: cloudsim-3.0.3.zip
 - Select Extract All
 - Choose a location (example): D:\cloudsim-3.0.3

Step 2: Launch Eclipse

- Open Eclipse IDE
- File → Switch Workspace → Other
- Choose: a location (D:\workspace_cloudsim)

Step 3: Create Java Project

- File → New → Java Project
- Project Name: CloudSimCustomScheduling
- Click Finish

Step 4: Add CloudSim JAR File

- Right-click Project → Properties
- Java Build Path → Libraries → Add External JARs
- Select: cloudsim-3.0.3.jar
- Click Apply and Close

Step 5: Create a new Java class

- Right-click src → New → Package
- Package Name: org.cloudbus.cloudsim.custom

Step 6: Create Custom Broker Class

- Right-click package
- Click New → Class

- Class Name: PriorityDatacenterBroker
- Click Finish

Coding:

```
package org.cloudbus.cloudsim.custom;

import java.util.Collections;
import java.util.Comparator;
import java.util.List;

import org.cloudbus.cloudsim.Cloudlet;
import org.cloudbus.cloudsim.DatacenterBroker;
import org.cloudbus.cloudsim.core.CloudSimTags;

public class PriorityDatacenterBroker extends DatacenterBroker {

    // REQUIRED constructor
    public PriorityDatacenterBroker(String name) throws Exception {
        super(name);
    }

    // Custom scheduling logic (NOT present in CloudSim)
    @Override
    protected void submitCloudlets() {

        List<Cloudlet> cloudletList = getCloudletList();

        // Sort cloudlets by length (higher length = higher priority)
        Collections.sort(cloudletList, new Comparator<Cloudlet>() {
            @Override
            public int compare(Cloudlet c1, Cloudlet c2) {
                return Long.compare(c2.getCloudletLength(), c1.getCloudletLength());
            }
        });

        for (Cloudlet cloudlet : cloudletList) {

            int vmIndex = cloudlet.getCloudletId() % getVmsCreatedList().size();
            cloudlet.setVmId(getVmsCreatedList().get(vmIndex).getId());
        }
    }
}
```

```

        sendNow(
            getVmsCreatedList().get(vmIndex).getId(),
            CloudSimTags.CLOUDLET_SUBMIT,
            cloudlet
        );
    }

    cloudletList.clear();
}
}

```

Step 6: Create Java Class

- Right-click Package - org.cloudbus.cloudsim.custom → New → Class
- Class Name: CustomSchedulingSimulation
- Check public static void main
- Click Finish

CODINGS:

```

package org.cloudbus.cloudsim.custom;

import java.util.*;

import org.cloudbus.cloudsim.*;

import org.cloudbus.cloudsim.core.CloudSim;

import org.cloudbus.cloudsim.provisioners.*;

public class CustomSchedulingSimulation {

    public static void main(String[] args) {

        try {

            int numUsers = 1;

            Calendar calendar = Calendar.getInstance();

            boolean traceFlag = false;

            CloudSim.init(numUsers, calendar, traceFlag);

            Datacenter datacenter = createDatacenter("Datacenter_1");

            PriorityDatacenterBroker broker =

```

```

        new PriorityDatacenterBroker("PriorityBroker");

        int brokerId = broker.getId();

        List<Vm> vmList = createVMs(brokerId, 2);

        List<Cloudlet> cloudletList = createCloudlets(brokerId, 5);

        broker.submitVmList(vmList);

        broker.submitCloudletList(cloudletList);

        CloudSim.startSimulation();

        List<Cloudlet> resultList = broker.getCloudletReceivedList();

        CloudSim.stopSimulation();

        printResults(resultList);

    } catch (Exception e) {

        e.printStackTrace();

    }

}

private static Datacenter createDatacenter(String name) {

    List<Host> hostList = new ArrayList<>();

    int mips = 1000;

    int ram = 8192;

    long storage = 1000000;

    int bw = 10000;

    List<Pe> peList = new ArrayList<>();

    peList.add(new Pe(0, new PeProvisionerSimple(mips)));

    hostList.add(new Host(

        0,

```

```

        new RamProvisionerSimple(ram),
        new BwProvisionerSimple(bw),
        storage,
        peList,
        new VmSchedulerTimeShared(peList)));
DatacenterCharacteristics characteristics =
    new DatacenterCharacteristics(
        "x86",
        "Linux",
        "Xen",
        hostList,
        10.0,
        3.0,
        0.05,
        0.1,
        0.1);
try {
    return new Datacenter(
        name,
        characteristics,
        new VmAllocationPolicySimple(hostList),
        new ArrayList<Storage>(),
        0);
} catch (Exception e) {
    e.printStackTrace();
}

```

```

    }

    return null;
}

private static List<Vm> createVMs(int brokerId, int vms) {
    List<Vm> list = new ArrayList<>();

    int mips = 1000;

    int ram = 1024;

    long size = 10000;

    int bw = 1000;

    int pesNumber = 1;

    String vmm = "Xen";

    for (int i = 0; i < vms; i++) {
        Vm vm = new Vm(
            i,
            brokerId,
            mips,
            pesNumber,
            ram,
            bw,
            size,
            vmm,
            new CloudletSchedulerTimeShared());

        list.add(vm);
    }

    return list;
}

```

```

    }

    private static List<Cloudlet> createCloudlets(int brokerId, int cloudlets) {

        List<Cloudlet> list = new ArrayList<>();

        long length;

        long fileSize = 300;

        long outputSize = 300;

        int pesNumber = 1;

        UtilizationModel utilizationModel =

            new UtilizationModelFull();

        for (int i = 0; i < cloudlets; i++) {

            length = (i + 1) * 1000; // priority increases with length

            Cloudlet cloudlet = new Cloudlet(

                i,

                length,

                pesNumber,

                fileSize,

                outputSize,

                utilizationModel,

                utilizationModel,

                utilizationModel);

            cloudlet.setUserId(brokerId);

            list.add(cloudlet);

        }

        return list;

    }

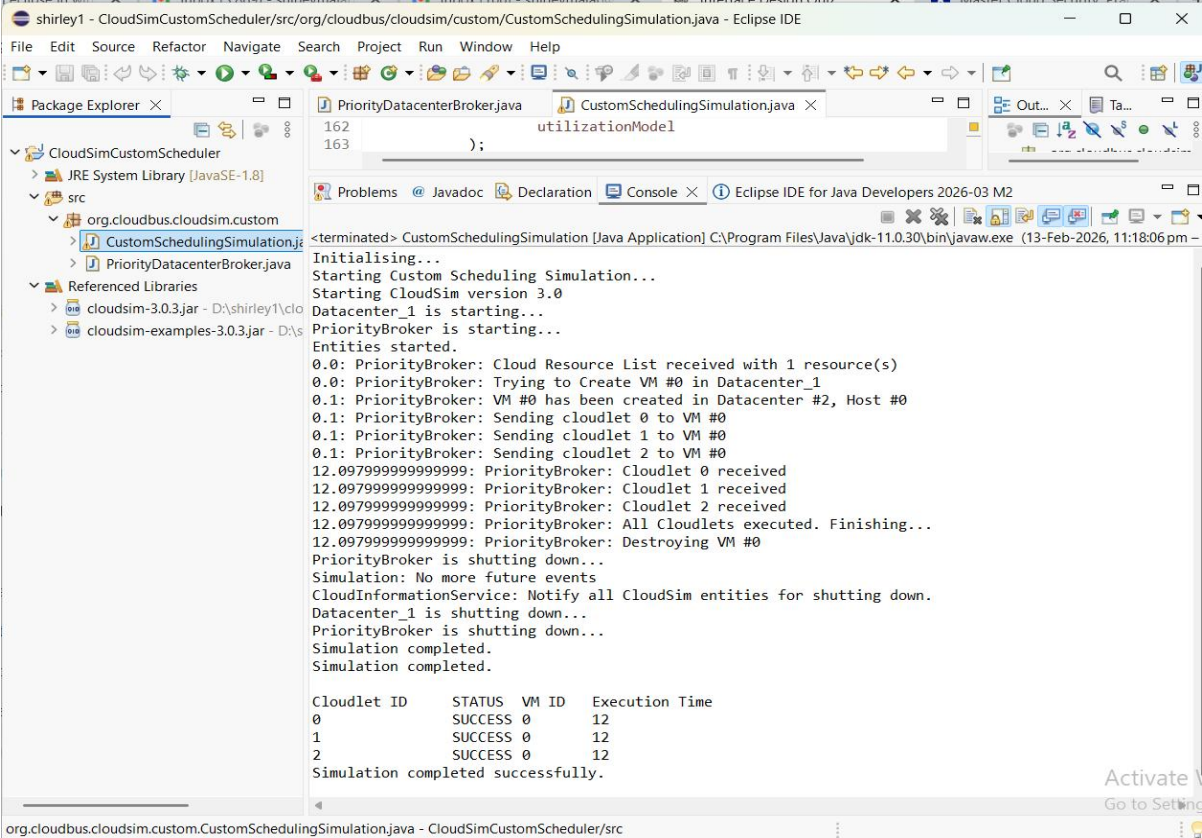
```

```
private static void printResults(List<Cloudlet> list) {  
    System.out.println("\nCloudlet ID\tSTATUS\tVM ID\tExecution Time");  
    for (Cloudlet c : list) {  
        System.out.println(  
            c.getCloudletId() + "\t\t" +  
            c.getStatus() + "\t" +  
            c.getVmId() + "\t" +  
            c.getActualCPUTime());  
    }  
}  
}
```

Step 7: Execution Steps

- Right-click CustomSchedulingSimulation.java
- Run As → Java Application

OUTPUT:



The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure: CloudSimCustomScheduler, JRE System Library [JavaSE-1.8], src, org.cloudbus.cloudsim.custom, CustomSchedulingSimulation.java, PriorityDatacenterBroker.java, and Referenced Libraries. The Console window on the right shows the following output:

```
<terminated> CustomSchedulingSimulation [Java Application] C:\Program Files\Java\jdk-11.0.30\bin\javaw.exe (13-Feb-2026, 11:18:06 pm -
Initialising...
Starting Custom Scheduling Simulation...
Starting CloudSim version 3.0
Datacenter_1 is starting...
PriorityBroker is starting...
Entities started.
0.0: PriorityBroker: Cloud Resource List received with 1 resource(s)
0.0: PriorityBroker: Trying to Create VM #0 in Datacenter_1
0.1: PriorityBroker: VM #0 has been created in Datacenter #2, Host #0
0.1: PriorityBroker: Sending cloudlet 0 to VM #0
0.1: PriorityBroker: Sending cloudlet 1 to VM #0
0.1: PriorityBroker: Sending cloudlet 2 to VM #0
12.097999999999999: PriorityBroker: Cloudlet 0 received
12.097999999999999: PriorityBroker: Cloudlet 1 received
12.097999999999999: PriorityBroker: Cloudlet 2 received
12.097999999999999: PriorityBroker: All Cloudlets executed. Finishing...
12.097999999999999: PriorityBroker: Destroying VM #0
PriorityBroker is shutting down...
Simulation: No more future events
CloudInformationService: Notify all CloudSim entities for shutting down.
Datacenter_1 is shutting down...
PriorityBroker is shutting down...
Simulation completed.
Simulation completed.

Cloudlet ID      STATUS  VM ID  Execution Time
0                SUCCESS 0      12
1                SUCCESS 0      12
2                SUCCESS 0      12
Simulation completed successfully.
```

RESULT:

Thus, a cloud computing environment was successfully simulated using CloudSim, and a custom scheduling algorithm was implemented by extending DatacenterBroker. Cloudlets were scheduled based on priority, and execution results were obtained.

Exp No:2	SIMULATE RESOURCE MANAGEMENT USING CLOUD SIM
Date:	

AIM:

To simulate resource management in a cloud computing environment using CloudSim by creating a datacenter, allocating virtual machines, scheduling cloudlets, and analyzing their execution performance.

PROCEDURE:**Step 1: Create a New Java Project in Eclipse**

- Open Eclipse
- File → New → Java Project
- Project Name: CloudSimResourceManagement
- Click Finish

Step 2 : Add CloudSim JAR Files

- Right-click project → Build Path
- Configure Build Path
- Libraries
- Add External JARs from : cloudsims-3.0.3/jars/
- Click Apply and Close

Step 3: Create Java Package & Class

- Right-click src
- New → Package - org.cloudsim.resource
- Right-click package → New → Class
- Name it as : ResourceManagementSimulation

CODING:

```
package org.cloudsim.resource;
```

```
import java.util.*;
```

```
import org.cloudbus.cloudsim.*;
```

```
import org.cloudbus.cloudsim.core.CloudSim;
```

```
import org.cloudbus.cloudsim.provisioners.*;
```

```
public class ResourceManagementSimulation {
```

```
public static void main(String[] args) {  
    try {  
        // Step 1: Initialize CloudSim  
        int numUsers = 1;  
        Calendar calendar = Calendar.getInstance();  
        boolean traceFlag = false;  
        CloudSim.init(numUsers, calendar, traceFlag);  
        // Step 2: Create Datacenter  
        Datacenter datacenter = createDatacenter("Datacenter_1");  
        // Step 3: Create Broker  
        DatacenterBroker broker = new DatacenterBroker("Broker_1");  
        int brokerId = broker.getId();  
        // Step 4: Create VMs  
        List<Vm> vmList = new ArrayList<>();  
        Vm vm1 = new Vm(  
            0, brokerId,  
            1000, // MIPS  
            1, // CPU cores  
            2048, // RAM  
            1000, // Bandwidth  
            10000, // Storage  
            "Xen",  
            new CloudletSchedulerTimeShared()  
        );  
        vmList.add(vm1);  
    }  
}
```

```

broker.submitVmList(vmList);

// Step 5: Create Cloudlets (Tasks)
List<Cloudlet> cloudletList = new ArrayList<>();

int length = 4000;

int fileSize = 300;

int outputSize = 300;

UtilizationModel model = new UtilizationModelFull();

Cloudlet cloudlet = new Cloudlet(
    0, length, 1,
    fileSize, outputSize,
    model, model, model
);

cloudlet.setUserId(brokerId);

cloudletList.add(cloudlet);

broker.submitCloudletList(cloudletList);

// Step 6: Start Simulation
CloudSim.startSimulation();

// Step 7: Stop Simulation
CloudSim.stopSimulation();

// Step 8: Print Results
printCloudletResults(broker.getCloudletReceivedList());
} catch (Exception e) {
    e.printStackTrace();
}
}

```

```
// Datacenter creation

private static Datacenter createDatacenter(String name) {

    List<Host> hostList = new ArrayList<>();

    List<Pe> peList = new ArrayList<>();

    peList.add(new Pe(0, new PeProvisionerSimple(1000)));

    int ram = 8192;

    long storage = 1000000;

    int bw = 10000;

    Host host = new Host(

        0,

        new RamProvisionerSimple(ram),

        new BwProvisionerSimple(bw),

        storage,

        peList,

        new VmSchedulerTimeShared(peList)

    );

    hostList.add(host);

    String arch = "x86";

    String os = "Linux";

    String vmm = "Xen";

    double timeZone = 10.0;

    double cost = 3.0;

    double costPerMem = 0.05;

    double costPerStorage = 0.001;
```

```

double costPerBw = 0.0;

DatacenterCharacteristics characteristics =

    new DatacenterCharacteristics(

        arch, os, vmm, hostList,

        timeZone, cost,

        costPerMem, costPerStorage, costPerBw

    );

Datacenter datacenter = null;

try {

    datacenter = new Datacenter(

        name,

        characteristics,

        new VmAllocationPolicySimple(hostList),

        new LinkedList<Storage>(),

        0

    );

} catch (Exception e) {

    e.printStackTrace();

}

return datacenter;

}

// Output results

private static void printCloudletResults(List<Cloudlet> list) {

    System.out.println("\n===== CLOUDLET EXECUTION RESULT =====");

```

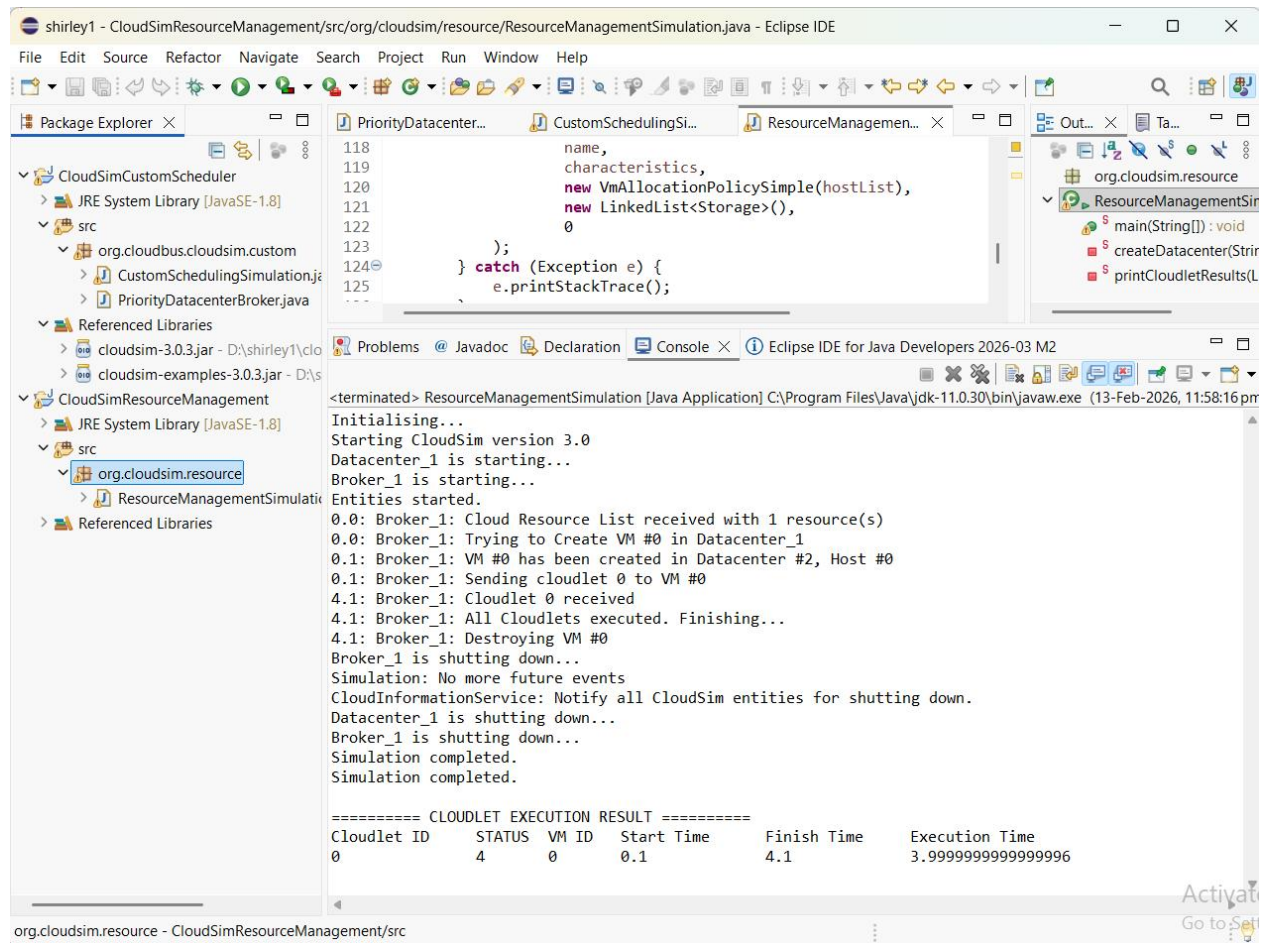
```
System.out.println("Cloudlet ID\tSTATUS\tVM ID\tStart Time\tFinish Time\tExecution  
Time");
```

```
for (Cloudlet cloudlet : list) {  
    System.out.println(  
        cloudlet.getCloudletId() + "\t\t" +  
        cloudlet.getStatus() + "\t" +  
        cloudlet.getVmId() + "\t" +  
        cloudlet.getExecStartTime() + "\t\t" +  
        cloudlet.getFinishTime() + "\t\t" +  
        (cloudlet.getFinishTime() - cloudlet.getExecStartTime())  
    );  
}  
}
```

Step 4: Run the Program

- Right-click ResourceManagementSimulation.java
- Run As → Java Application

OUTPUT:



The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure for 'CloudSimResourceManagement'. The main editor window shows the 'ResourceManagementSimulation.java' file with the following code snippet:

```
118     name,  
119     characteristics,  
120     new VmAllocationPolicySimple(hostList),  
121     new LinkedList<Storage>(),  
122     0  
123 );  
124 } catch (Exception e) {  
125     e.printStackTrace();  
126 }
```

The Console window at the bottom displays the simulation output:

```
<terminated> ResourceManagementSimulation [Java Application] C:\Program Files\Java\jdk-11.0.30\bin\javaw.exe (13-Feb-2026, 11:58:16 pm)  
Initialising...  
Starting CloudSim version 3.0  
Datacenter_1 is starting...  
Broker_1 is starting...  
Entities started.  
0.0: Broker_1: Cloud Resource List received with 1 resource(s)  
0.0: Broker_1: Trying to Create VM #0 in Datacenter_1  
0.1: Broker_1: VM #0 has been created in Datacenter #2, Host #0  
0.1: Broker_1: Sending cloudlet 0 to VM #0  
4.1: Broker_1: Cloudlet 0 received  
4.1: Broker_1: All Cloudlets executed. Finishing...  
4.1: Broker_1: Destroying VM #0  
Broker_1 is shutting down...  
Simulation: No more future events  
CloudInformationService: Notify all CloudSim entities for shutting down.  
Datacenter_1 is shutting down...  
Broker_1 is shutting down...  
Simulation completed.  
Simulation completed.
```

Below the console output, a table titled 'CLOUDLET EXECUTION RESULT' is displayed:

Cloudlet ID	STATUS	VM ID	Start Time	Finish Time	Execution Time
0	4	0	0.1	4.1	3.9999999999999996

RESULT:

The cloud computing environment was successfully simulated using CloudSim. The virtual machine was allocated to the host, cloudlets were scheduled and executed using the Time-Shared policy, and the execution details such as start time, finish time, and execution time were obtained successfully, demonstrating effective resource management.

Exp No:3	SIMULATE LOG FORENSICS USING CLOUD SIM
Date:	

AIM:

To simulate log forensics in a cloud computing environment using CloudSim by generating execution logs from cloudlets and analyzing them to detect abnormal behavior.

PROCEDURE:

Step 1: Create a Java Project

- Open Eclipse IDE.
- Click File → New → Java Project.
- Enter the project name as: CloudSimLogForensics
- Select JavaSE-1.8 as the JRE.
- Click Finish.

Step 2: Add CloudSim JAR Files

- Right-click on the project CloudSimLogForensics.
- Select Build Path → Configure Build Path.
- Go to the Libraries tab.
- Click Add External JARs.
- Browse to the folder: cloudsims-3.0.3/jars
- Select all JAR files and click Open.
- Click Apply and Close.

Step 3: Create Package

- Right-click on src folder.
- Select New → Package.
- Enter the package name as: org.cloudsim.logforensics
- Click Finish.

Step 4: Create Java Class

- Right-click on the created package.
- Select New → Class.
- Enter the class name as: LogForensicsSimulation
- Select:
 - Modifier: public
 - Superclass: java.lang.Object
 - Check ☒ public static void main(String[] args)

- Click Finish.

CODINGS:

Open LogForensicsSimulation.java.

```
package org.cloudsim.logforensics;

import java.util.*;

import org.cloudbus.cloudsim.*;

import org.cloudbus.cloudsim.core.CloudSim;

import org.cloudbus.cloudsim.provisioners.*;

public class LogForensicsSimulation {

    public static void main(String[] args) {

        try {

            // Step 1: Initialize CloudSim

            CloudSim.init(1, Calendar.getInstance(), false);

            // Step 2: Create Datacenter

            Datacenter datacenter = createDatacenter("Forensic_Datacenter");

            // Step 3: Create Broker

            DatacenterBroker broker = new DatacenterBroker("Forensic_Broker");

            int brokerId = broker.getId();

            // Step 4: Create VM

            Vm vm = new Vm(

                0, brokerId,

                1000, 1,

                2048, 1000,

                10000, "Xen",

                new CloudletSchedulerTimeShared()
```

```

);
broker.submitVmList(Arrays.asList(vm));

// Step 5: Create Cloudlets (Simulated Log Sources)
List<Cloudlet> cloudletList = new ArrayList<>();
UtilizationModel model = new UtilizationModelFull();
for (int i = 0; i < 5; i++) {
    Cloudlet cloudlet = new Cloudlet(
        i,
        (i + 1) * 2000, // Different execution times
        1, 300, 300,
        model, model, model
    );
    cloudlet.setUserId(brokerId);
    cloudletList.add(cloudlet);
}
broker.submitCloudletList(cloudletList);

// Step 6: Start Simulation
CloudSim.startSimulation();
CloudSim.stopSimulation();

// Step 7: Log Forensics Analysis
performLogForensics(broker.getCloudletReceivedList());
} catch (Exception e) {
    e.printStackTrace();
}
}

```

```
// Datacenter Creation
```

```
private static Datacenter createDatacenter(String name) {
```

```
    List<Host> hostList = new ArrayList<>();
```

```
    List<Pe> peList = new ArrayList<>();
```

```
    peList.add(new Pe(0, new PeProvisionerSimple(1000)));
```

```
    Host host = new Host(
```

```
        0,
```

```
        new RamProvisionerSimple(8192),
```

```
        new BwProvisionerSimple(10000),
```

```
        1000000,
```

```
        peList,
```

```
        new VmSchedulerTimeShared(peList)
```

```
    );
```

```
    hostList.add(host);
```

```
    DatacenterCharacteristics characteristics =
```

```
        new DatacenterCharacteristics(
```

```
            "x86", "Linux", "Xen",
```

```
            hostList, 10.0,
```

```
            3.0, 0.05, 0.001, 0.0
```

```
        );
```

```
    try {
```

```
        return new Datacenter(
```

```
            name,
```

```

        characteristics,
        new VmAllocationPolicySimple(hostList),
        new LinkedList<Storage>(),
        0
    );
} catch (Exception e) {
    return null;
}
}

// Log Forensics Function
private static void performLogForensics(List<Cloudlet> logs) {
    System.out.println("\n===== LOG FORENSICS REPORT =====");
    System.out.println("ID\tSTATUS\tEXEC_TIME\tANALYSIS");
    double threshold = 5.0; // forensic threshold
    for (Cloudlet c : logs) {
        double execTime = c.getFinishTime() - c.getExecStartTime();
        String analysis;
        if (execTime > threshold) {
            analysis = "⚠ Suspicious Delay Detected";
        } else {
            analysis = "Normal Execution";
        }
        System.out.println(
            c.getCloudletId() + "\t" +
            c.getStatus() + "\t" +

```

```

        execTime + "\t\t" +

        analysis

    );

}

}

}

```

OUTPUT:

```

<terminated> LogForensicsSimulation [Java Application] C:\Program Files\Java\jdk-11.0.30\bin\javaw.exe (14-Feb-2026, 1:17:32 am - 1:17:33
Entities started.
0.0: Forensic_Broker: Cloud Resource List received with 1 resource(s)
0.0: Forensic_Broker: Trying to Create VM #0 in Forensic_Datacenter
0.1: Forensic_Broker: VM #0 has been created in Datacenter #2, Host #0
0.1: Forensic_Broker: Sending cloudlet 0 to VM #0
0.1: Forensic_Broker: Sending cloudlet 1 to VM #0
0.1: Forensic_Broker: Sending cloudlet 2 to VM #0
0.1: Forensic_Broker: Sending cloudlet 3 to VM #0
0.1: Forensic_Broker: Sending cloudlet 4 to VM #0
10.1: Forensic_Broker: Cloudlet 0 received
18.1: Forensic_Broker: Cloudlet 1 received
24.1: Forensic_Broker: Cloudlet 2 received
28.1: Forensic_Broker: Cloudlet 3 received
30.1: Forensic_Broker: Cloudlet 4 received
30.1: Forensic_Broker: All Cloudlets executed. Finishing...
30.1: Forensic_Broker: Destroying VM #0
Forensic_Broker is shutting down...
Simulation: No more future events
CloudInformationService: Notify all CloudSim entities for shutting down.
Forensic_Datacenter is shutting down...
Forensic_Broker is shutting down...
Simulation completed.
Simulation completed.

===== LOG FORENSICS REPORT =====
ID      STATUS  EXEC_TIME  ANALYSIS
0       4       10.0      Δ Suspicious Delay Detected
1       4       18.0      Δ Suspicious Delay Detected
2       4       24.0      Δ Suspicious Delay Detected
3       4       28.0      Δ Suspicious Delay Detected
4       4       30.0      Δ Suspicious Delay Detected

```

RESULT:

The log forensics simulation was successfully executed using CloudSim. Execution logs were generated and analyzed to identify normal and abnormal behavior, demonstrating the effectiveness of cloud log monitoring and forensic analysis.

Exp No:4	SIMULATION OF SECURE FILE SHARING IN CLOUD COMPUTING USING CLOUDSIM
Date:	

AIM:

To simulate secure file sharing in a cloud environment using CloudSim by controlling access to virtual machines, executing secure file transfer tasks, and monitoring execution logs.

PROCEDURE:

STEP 1: Create a Java Project

- Open Eclipse IDE
- Click File → New → Java Project
- Enter Project Name: CloudSimSecureFileSharing
- Select JavaSE-1.8
- Click Finish

STEP 2: Add CloudSim JAR Files

- Right-click CloudSimSecureFileSharing
- Select Build Path → Configure Build Path
- Open Libraries tab
- Click Add External JARs
- Select all JARs from
- Click Open → Apply → Apply and Close

STEP 3: Create Package

- Right-click src
- Select New → Package
- Package name: org.cloudsim.securefilesharing
- Click Finish

STEP 4: Create Java Class

- Right-click the package
- Select New → Class
- Class name: SecureFileSharingSimulation
- Choose:
 - Modifier: public
 - Superclass: java.lang.Object
 - ☒ public static void main(String[] args)

- Click Finish

CODINGS:

```
package org.cloudsim.securefilesharing;

import java.util.*;

import org.cloudbus.cloudsim.*;

import org.cloudbus.cloudsim.core.CloudSim;

import org.cloudbus.cloudsim.provisioners.*;

public class SecureFileSharingSimulation {

    public static void main(String[] args) {

        try {

            // Step 1: Initialize CloudSim

            CloudSim.init(1, Calendar.getInstance(), false);

            // Step 2: Create Datacenter

            Datacenter datacenter = createDatacenter("Secure_Datacenter");

            // Step 3: Create Broker (Authenticated User)

            DatacenterBroker broker = new DatacenterBroker("Secure_Broker");

            int brokerId = broker.getId();

            // Step 4: Create Secure VM

            Vm secureVm = new Vm(

                0, brokerId,

                1000, 1,

                2048, 1000,

                10000, "Xen",

                new CloudletSchedulerTimeShared()

            );

            broker.submitVmList(Arrays.asList(secureVm));
```



```

// Step 5: Create Secure File Transfer (Cloudlet)

List<Cloudlet> cloudletList = new ArrayList<>();

UtilizationModel model = new UtilizationModelFull();

Cloudlet secureFile = new Cloudlet(

    0,

    3000,    // file size / work length

    1,

    300, 300,

    model, model, model

);

secureFile.setUserId(brokerId);

cloudletList.add(secureFile);

broker.submitCloudletList(cloudletList);

// Step 6: Start Simulation

CloudSim.startSimulation();

CloudSim.stopSimulation();

// Step 7: Secure Access Log

printSecureLog(broker.getCloudletReceivedList());

} catch (Exception e) {

    e.printStackTrace();

}

}

// Datacenter Creation

private static Datacenter createDatacenter(String name) {

```

```

List<Host> hostList = new ArrayList<>();

List<Pe> peList = new ArrayList<>();

peList.add(new Pe(0, new PeProvisionerSimple(1000)));

Host host = new Host(
    0,
    new RamProvisionerSimple(8192),
    new BwProvisionerSimple(10000),
    1000000,
    peList,
    new VmSchedulerTimeShared(peList)
);

hostList.add(host);

DatacenterCharacteristics characteristics =
    new DatacenterCharacteristics(
        "x86", "Linux", "Xen",
        hostList, 10.0,
        3.0, 0.05, 0.001, 0.0
    );

try {
    return new Datacenter(
        name,
        characteristics,
        new VmAllocationPolicySimple(hostList),
        new LinkedList<Storage>(),
        0
    );
}

```

```

    } catch (Exception e) {

        return null;

    }

}

// Secure Log Output
private static void printSecureLog(List<Cloudlet> list) {

    System.out.println("\n===== SECURE FILE SHARING LOG =====");

    System.out.println("FileID\tSTATUS\tVM\tACCESS");

    for (Cloudlet c : list) {

        System.out.println(

            c.getCloudletId() + "\t" +

            c.getStatus() + "\t" +

            c.getVmId() + "\tAuthorized Access"

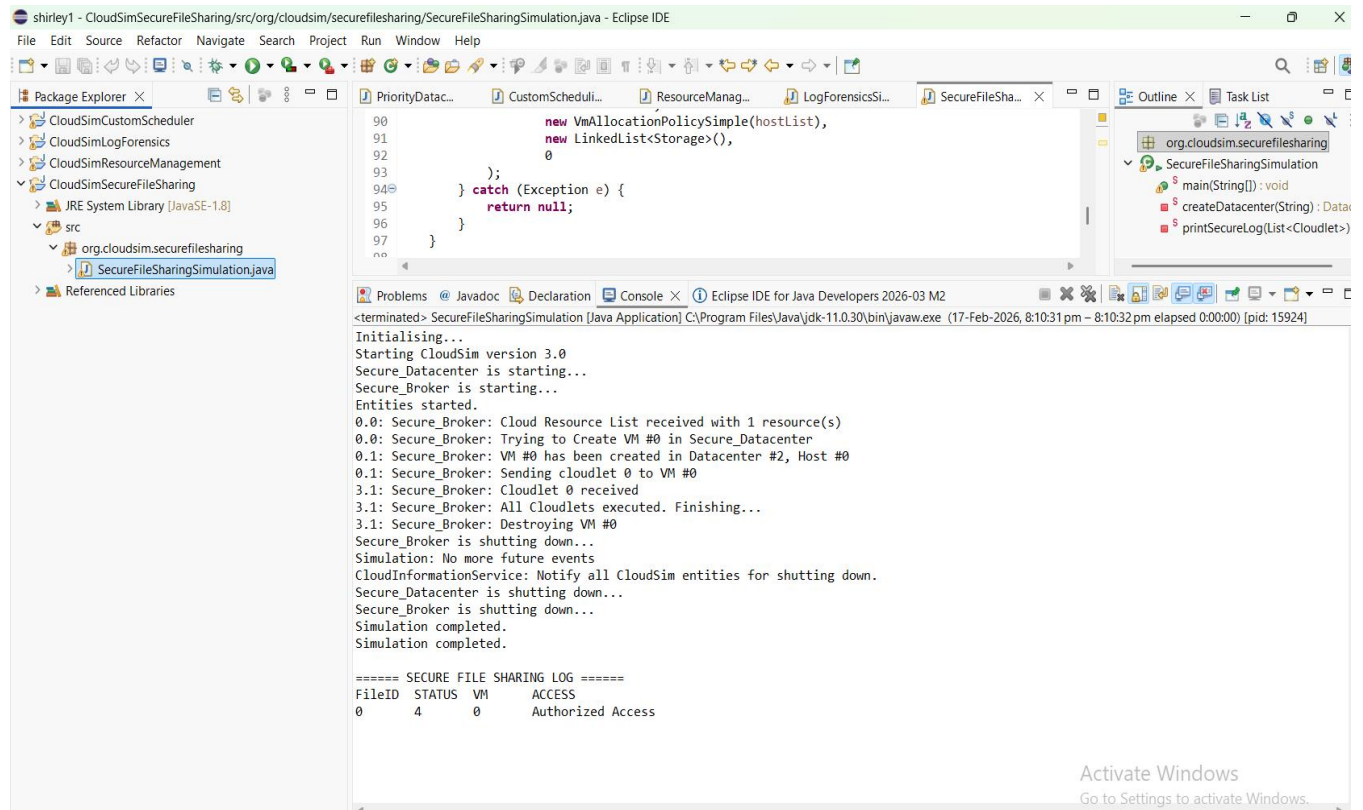
        );

    }

}
}

```

OUTPUT:



RESULT:

The secure file sharing simulation was successfully executed using CloudSim. Files were securely transferred through authorized virtual machines, and access logs confirmed controlled and secure cloud file sharing.

Exp No:5	IMPLEMENT DATA ANONYMIZATION TECHNIQUES OVER THE SIMPLE DATASET (MASKING, KANONYMIZATION, ETC)
Date:	

AIM:

To implement data anonymization techniques such as Data Masking and k-Anonymity on a simple dataset to protect sensitive information while preserving data utility.

PROCEDURE:

Step 1: Create Java Project: DataAnonymization

Step 2: Create Java Class

- Class Name: DataAnonymizationDemo

CODINGS:

```
import java.util.*;
```

```
class Record {
```

```
    int id;
```

```
    String name;
```

```
    int age;
```

```
    String gender;
```

```
    String zip;
```

```
    String disease;
```

```
    Record(int id, String name, int age, String gender, String zip, String disease) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
        this.age = age;
```

```
        this.gender = gender;
```

```
        this.zip = zip;
```

```
        this.disease = disease;
```

```
    }
```

```
}
```

```
public class DataAnonymizationDemo {
```

```
    public static void main(String[] args) {
```

```
        // Original Dataset
```

```
        List<Record> dataset = Arrays.asList(
```

```
            new Record(1, "Alice", 28, "F", "560001", "Flu"),
```

```
            new Record(2, "Bob", 29, "M", "560002", "Diabetes"),
```

```
            new Record(3, "Carol", 27, "F", "560001", "Flu"),
```

```
            new Record(4, "David", 28, "M", "560002", "Cancer")
```

```

);

System.out.println("ORIGINAL DATASET");
printDataset(dataset);

System.out.println("\nAFTER DATA MASKING");
applyMasking(dataset);

System.out.println("\nAFTER k-ANONYMITY (k = 2)");
applyKAnonymity(dataset);
}

// Print dataset
static void printDataset(List<Record> data) {
    System.out.println("ID\tName\tAge\tGender\tZIP\tDisease");
    for (Record r : data) {
        System.out.println(
            r.id + "\t" + r.name + "\t" + r.age + "\t" +
            r.gender + "\t" + r.zip + "\t" + r.disease
        );
    }
}

// Data Masking
static void applyMasking(List<Record> data) {
    System.out.println("ID\tName\tAge\tGender\tZIP\tDisease");
    for (Record r : data) {
        String maskedName = r.name.charAt(0) + "*****";
        String maskedZip = r.zip.substring(0, 3) + "****";

        System.out.println(
            r.id + "\t" +
            maskedName + "\t" +
            r.age + "\t" +
            r.gender + "\t" +
            maskedZip + "\t" +
            r.disease
        );
    }
}

```

```
// k-Anonymity (k = 2)
static void applyKAnonymity(List<Record> data) {
    System.out.println("Age\tGender\tZIP\tDisease");

    for (Record r : data) {
        String ageGroup = (r.age / 10) * 10 + "-" + ((r.age / 10) * 10 + 9);
        String zipGroup = r.zip.substring(0, 3) + "****";

        System.out.println(
            ageGroup + "\t" +
            r.gender + "\t" +
            zipGroup + "\t" +
            r.disease
        );
    }
}
```

OUTPUT:

The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays a project named 'DataAnonymization' with a source folder 'src' containing 'DataAnonymizationDemo.java'. The main editor shows the code for 'DataAnonymizationDemo.java', which includes a loop to print records and a static method 'applyMasking' that masks data. The Console at the bottom shows the output of the program, displaying the original dataset, the dataset after data masking, and the result of k-anonymity (k=2).

```
45 System.out.println("ID\tName\tAge\tGender\tZIP\tDisease");
46 for (Record r : data) {
47     System.out.println(
48         r.id + "\t" + r.name + "\t" + r.age + "\t" +
49         r.gender + "\t" + r.zip + "\t" + r.disease
50     );
51 }
52 }
53 }
54 // Data Masking
55 static void applyMasking(List<Record> data) {
56     System.out.println("ID\tName\tAge\tGender\tZIP\tDisease");
```

ORIGINAL DATASET

ID	Name	Age	Gender	ZIP	Disease
1	Alice	28	F	560001	Flu
2	Bob	29	M	560002	Diabetes
3	Carol	27	F	560001	Flu
4	David	28	M	560002	Cancer

AFTER DATA MASKING

ID	Name	Age	Gender	ZIP	Disease
1	A****	28	F	560***	Flu
2	B****	29	M	560***	Diabetes
3	C****	27	F	560***	Flu
4	D****	28	M	560***	Cancer

AFTER k-ANONYMITY (k = 2)

Age	Gender	ZIP	Disease
20-29	F	560***	Flu
20-29	M	560***	Diabetes
20-29	F	560***	Flu
20-29	M	560***	Cancer

RESULT:

The data anonymization techniques were successfully implemented on a simple dataset. Data masking protected direct identifiers, and k-anonymity generalized quasi-identifiers, thereby ensuring privacy preservation while maintaining data usability.

Exp No:6	IMPLEMENT ANY ENCRYPTION ALGORITHM TO PROTECT THE IMAGES
Date:	

AIM:

To implement an image encryption and decryption system using the Advanced Encryption Standard (AES) algorithm in Java in order to protect image data from unauthorized access.

ALGORITHM:

- Open Eclipse IDE
- Create a new Java Project and name it as ImageEncryption.
- Right-click on the project → New → Class.
- Name the class as ImageEncryptionAES.
- Place the input image file input.jfif inside the project directory.
- Declare the AES algorithm and secret key inside the class.
- Read the input image file using FileInputStream.
- Convert the image into a byte array.
- Initialize the AES cipher in ENCRYPT mode.
- Encrypt the image byte array using AES.
- Store the encrypted data into a file named encrypted.img.
- Initialize the AES cipher in DECRYPT mode.
- Read the encrypted file using FileInputStream.
- Decrypt the encrypted byte array using the same AES key.
- Store the decrypted image as decrypted.jpg.
- Run the Java program.

PROGRAM CODE:

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.io.*;
import java.util.Arrays;

public class ImageEncryptionAES {

    private static final String ALGORITHM = "AES";
    private static final byte[] keyValue =
        "1234567890123456".getBytes(); // 16-byte key

    public static void main(String[] args) throws Exception {

        File inputFile = new File("input.jfif");
        File encryptedFile = new File("encrypted.img");
```

```

File decryptedFile = new File("decrypted.jpg");

encryptImage(inputFile, encryptedFile);
decryptImage(encryptedFile, decryptedFile);

System.out.println("Image encryption and decryption completed.");
}

private static void encryptImage(File input, File output) throws Exception {
    byte[] imageBytes = readFile(input);
    Cipher cipher = Cipher.getInstance(ALGORITHM);
    cipher.init(Cipher.ENCRYPT_MODE, generateKey());
    byte[] encrypted = cipher.doFinal(imageBytes);
    writeFile(output, encrypted);
}

private static void decryptImage(File input, File output) throws Exception {
    byte[] encryptedBytes = readFile(input);
    Cipher cipher = Cipher.getInstance(ALGORITHM);
    cipher.init(Cipher.DECRYPT_MODE, generateKey());
    byte[] decrypted = cipher.doFinal(encryptedBytes);
    writeFile(output, decrypted);
}

private static SecretKeySpec generateKey() {
    return new SecretKeySpec(Arrays.copyOf(keyValue, 16), ALGORITHM);
}

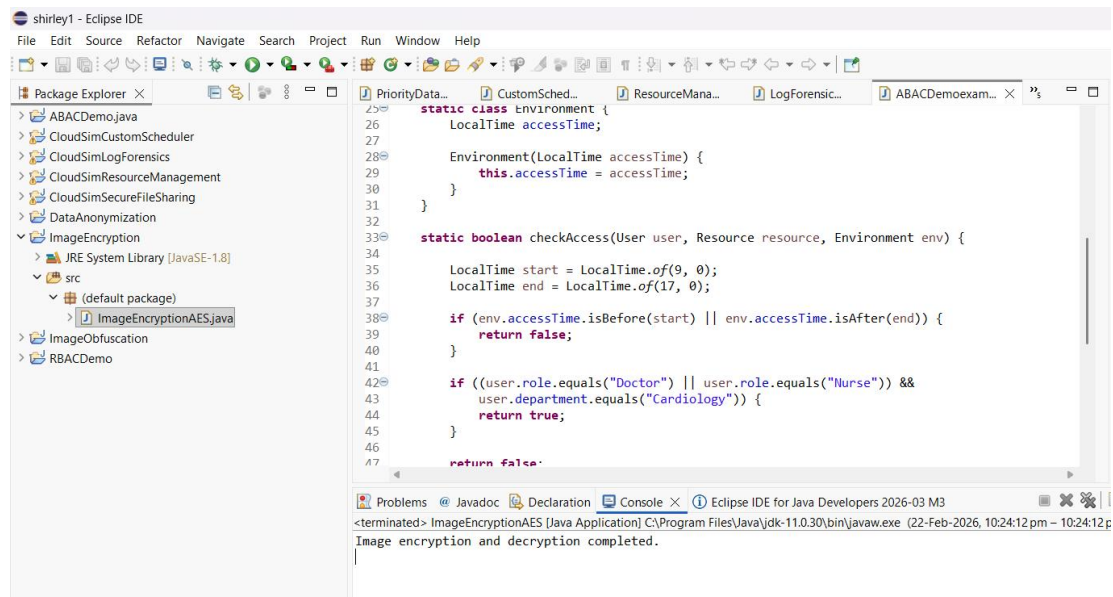
private static byte[] readFile(File file) throws IOException {
    FileInputStream fis = new FileInputStream(file);
    byte[] data = new byte[(int) file.length()];
    fis.read(data);
    fis.close();
    return data;
}

private static void writeFile(File file, byte[] data) throws IOException {
    FileOutputStream fos = new FileOutputStream(file);
    fos.write(data);
    fos.close();
}

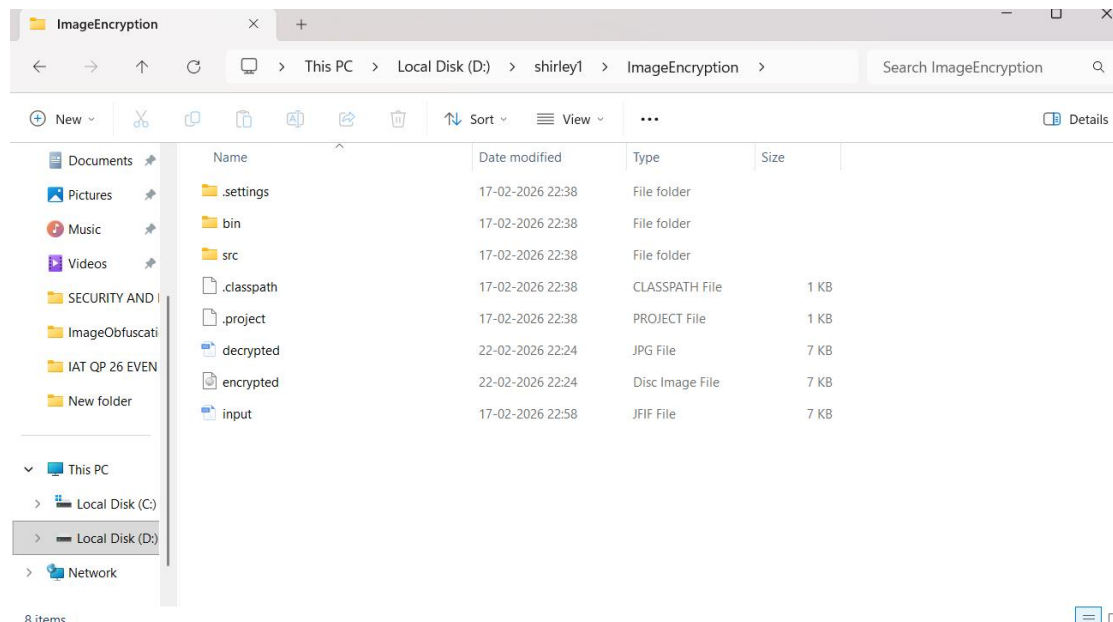
```

```
}  
}
```

OUTPUT:



```
shirley1 - Eclipse IDE  
File Edit Source Refactor Navigate Search Project Run Window Help  
Package Explorer  
ABACDemo.java  
CloudSimCustomScheduler  
CloudSimLogForensics  
CloudSimResourceManagement  
CloudSimSecureFileSharing  
DataAnonymization  
ImageEncryption  
  JRE System Library [JavaSE-1.8]  
  src  
    (default package)  
    ImageEncryptionAES.java  
ImageObfuscation  
RBACDemo  
PriorityData... CustomSched... ResourceMana... LogForensic... ABACDemoexam...  
25  
26 static class environment {  
27     LocalTime accessTime;  
28  
29     Environment(LocalTime accessTime) {  
30         this.accessTime = accessTime;  
31     }  
32  
33 static boolean checkAccess(User user, Resource resource, Environment env) {  
34  
35     LocalTime start = LocalTime.of(9, 0);  
36     LocalTime end = LocalTime.of(17, 0);  
37  
38     if (env.accessTime.isBefore(start) || env.accessTime.isAfter(end)) {  
39         return false;  
40     }  
41  
42     if ((user.role.equals("Doctor") || user.role.equals("Nurse")) &&  
43         user.department.equals("Cardiology")) {  
44         return true;  
45     }  
46  
47     return false;  
Problems @ Javadoc Declaration Console X Eclipse IDE for Java Developers 2026-03 M3  
<terminated> ImageEncryptionAES [Java Application] C:\Program Files\Java\jdk-11.0.30\bin\javaw.exe (22-Feb-2026, 10:24:12 p  
Image encryption and decryption completed.
```



Name	Date modified	Type	Size
.settings	17-02-2026 22:38	File folder	
bin	17-02-2026 22:38	File folder	
src	17-02-2026 22:38	File folder	
.classpath	17-02-2026 22:38	CLASSPATH File	1 KB
.project	17-02-2026 22:38	PROJECT File	1 KB
decrypted	22-02-2026 22:24	JPG File	7 KB
encrypted	22-02-2026 22:24	Disc Image File	7 KB
input	17-02-2026 22:58	JFIF File	7 KB

RESULT:

Thus, AES encryption and decryption was successfully implemented using Java to securely protect image data. The confidentiality of the image was ensured by converting it into encrypted form and restoring it back without any data loss.

Exp No:7	IMPLEMENT ANY IMAGE OBFUSCATION MECHANISM
Date:	

AIM:

To implement an image obfuscation mechanism by modifying pixel values so that the original image becomes visually unreadable while still being reversible.

PROCEDURE:

- Open Eclipse IDE.
- Create a new Java Project named ImageObfuscation.
- Right-click on the project → New → Class.
- Name the class as ImageObfuscationDemo.
- Place the input image file (e.g., input.jpg or input.jfif) inside the project folder.
- Read the image using ImageIO.read().
- Get the width and height of the image.
- Traverse each pixel using nested loops.
- Extract RGB values from each pixel.
- Obfuscate the image by inverting RGB values using 255 - value.
- Set the modified pixel back into the image.
- Save the obfuscated image as obfuscated.png.
- Run the program.

PROGRAM CODE:

```
import java.awt.image.BufferedImage;

import java.io.File;

import javax.imageio.ImageIO;

public class ImageObfuscationDemo {

    public static void main(String[] args) {

        try {

            // Step 1: Read input image

            BufferedImage image = ImageIO.read(new File("input.jfif"));
```

```
int width = image.getWidth();

int height = image.getHeight();

// Step 2: Obfuscate image by inverting pixel values

for (int y = 0; y < height; y++) {

    for (int x = 0; x < width; x++) {

        int pixel = image.getRGB(x, y);

        int alpha = (pixel >> 24) & 0xff;

        int red   = (pixel >> 16) & 0xff;

        int green = (pixel >> 8) & 0xff;

        int blue  = pixel & 0xff;

        // Obfuscation logic

        red   = 255 - red;

        green = 255 - green;

        blue  = 255 - blue;

        int obfuscatedPixel =

            (alpha << 24) | (red << 16) | (green << 8) | blue;

        image.setRGB(x, y, obfuscatedPixel);

    }

}

// Step 3: Save obfuscated image

ImageIO.write(image, "png", new File("obfuscated.png"));

System.out.println("Image obfuscation completed successfully.");
```

```

    } catch (Exception e) {

        e.printStackTrace();

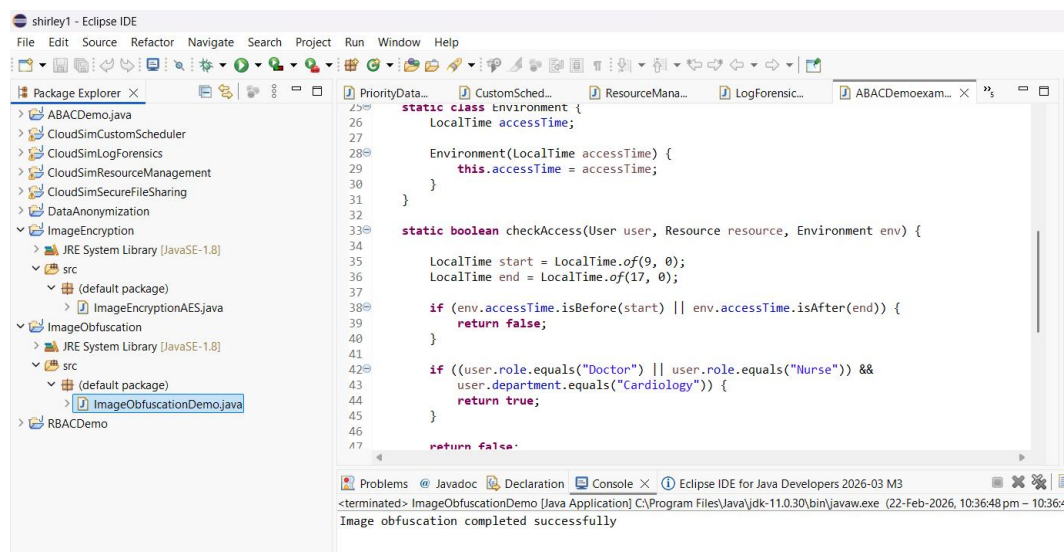
    }

}

}

```

OUTPUT:



The screenshot shows the Eclipse IDE interface. The Package Explorer on the left lists several packages, including 'ImageObfuscation'. The main editor displays the 'ImageObfuscationDemo.java' file. The code defines a static class 'Environment' with a 'LocalTime accessTime' attribute and a constructor. It also includes a 'checkAccess' method that checks if the current time is within a specified range and if the user has the correct role and department. The console at the bottom shows the message: 'Image obfuscation completed successfully'.

```

25  static class Environment {
26      LocalTime accessTime;
27
28      Environment(LocalTime accessTime) {
29          this.accessTime = accessTime;
30      }
31  }
32
33  static boolean checkAccess(User user, Resource resource, Environment env) {
34
35      LocalTime start = LocalTime.of(9, 0);
36      LocalTime end = LocalTime.of(17, 0);
37
38      if (env.accessTime.isBefore(start) || env.accessTime.isAfter(end)) {
39          return false;
40      }
41
42      if ((user.role.equals("Doctor") || user.role.equals("Nurse")) &&
43          user.department.equals("Cardiology")) {
44          return true;
45      }
46
47      return false;

```



RESULT:

Thus, an image obfuscation mechanism using pixel inversion was successfully implemented. The technique effectively hides visual information while maintaining reversibility.

Exp No:8	IMPLEMENT A ROLE-BASED ACCESS CONTROL MECHANISM IN A SPECIFIC SCENARIO
Date:	

AIM:

To implement a Role-Based Access Control (RBAC) mechanism for a file access scenario using Java, where permissions are granted to users based on their assigned roles.

PROCEDURE:

Step 1: Start the IDE

Step 2: Create a New Java Project

- Click on File → New → Java Project
- Enter the Project Name as: RBACDemo
- Click Finish

Step 3: Create a Java Class

- Right-click on the src folder
- Select New → Class
- Enter Class Name: RBACDemo
- Check the option: public static void main(String[] args)
- Click Finish

Step 4: Define User Roles

Step 5: Create User Class

Step 6: Define Role Permissions

Step 7: Implement Access Control Logic

Step 8: Simulate Users

Step 10: Compile the Program

PROGRAM CODE:

```
public class RBACExample {
```

```
    // Step 1: Define Roles
```

```
    enum Role {
```

```
        ADMIN,
```

```
        USER,

        GUEST
    }

// Step 2: Create User class
static class User {

    String username;

    Role role;

    User(String username, Role role) {

        this.username = username;

        this.role = role;

    }

}

// Step 3: Access Control Method
static void checkAccess(User user, String action) {

    System.out.print(user.username + " (" + user.role + ") -> ");

    switch (user.role) {

        case ADMIN:

            System.out.println("Access GRANTED for " + action);

            break;

        case USER:

            if (action.equalsIgnoreCase("delete")) {

                System.out.println("Access DENIED for " + action);

            } else {

                System.out.println("Access GRANTED for " + action);

            }

    }

}
```



```

        break;

    case GUEST:

        if (action.equalsIgnoreCase("view")) {

            System.out.println("Access GRANTED for " + action);

        } else {

            System.out.println("Access DENIED for " + action);

        }

        break;

    }

}

// Step 4: Main Method

public static void main(String[] args) {

    // Create users with different roles

    User admin = new User("Alice", Role.ADMIN);

    User user = new User("Bob", Role.USER);

    User guest = new User("Charlie", Role.GUEST);

    System.out.println("----- ROLE BASED ACCESS CONTROL -----");

    // Admin actions

    checkAccess(admin, "upload");

    checkAccess(admin, "download");

    checkAccess(admin, "delete");

    // User actions

    checkAccess(user, "upload");

    checkAccess(user, "download");

    checkAccess(user, "delete");

```

```

// Guest actions

checkAccess(guest, "view");

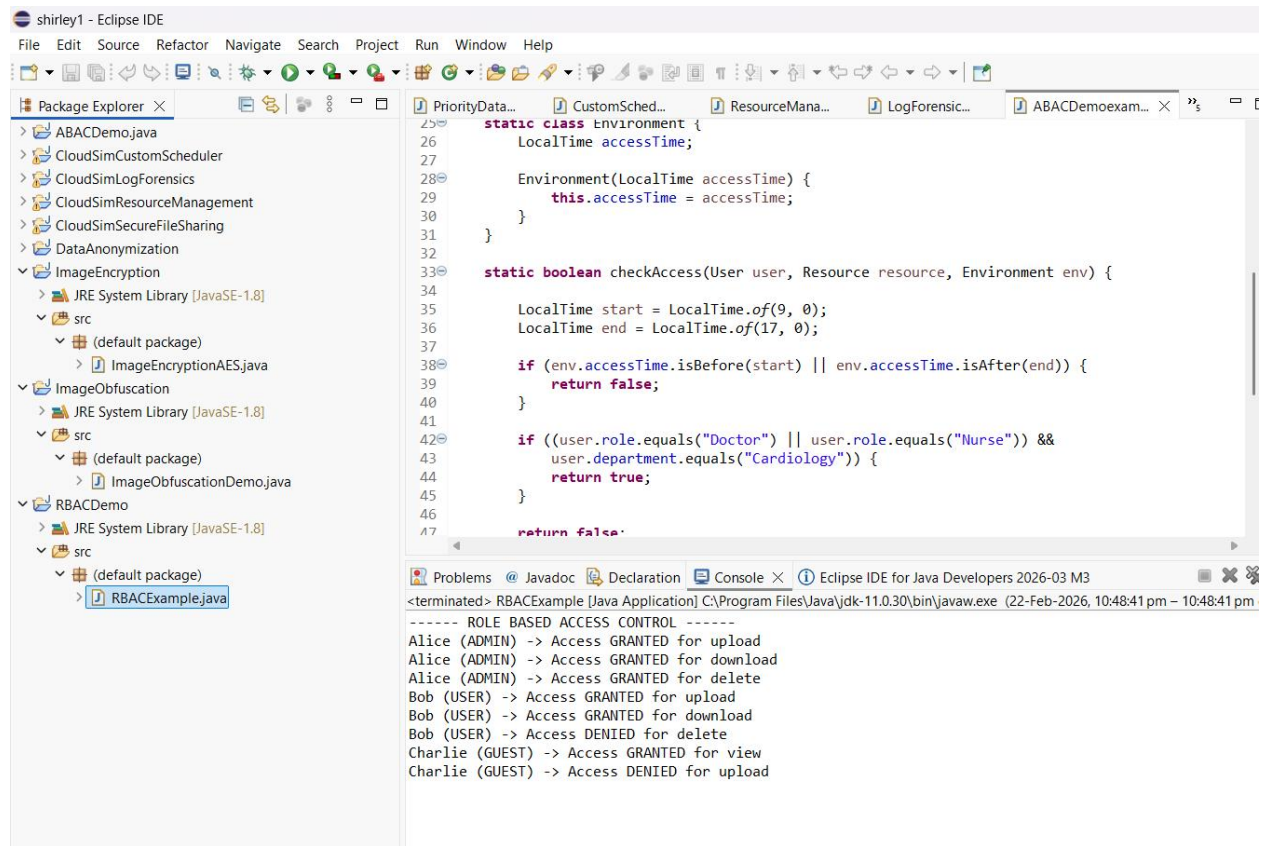
checkAccess(guest, "upload");

}

}

```

OUTPUT:



The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays a project structure with packages like ABACDemojava, CloudSimCustomScheduler, CloudSimLogForensics, CloudSimResourceManagement, CloudSimSecureFileSharing, DataAnonymization, ImageEncryption, ImageObfuscation, and RBACDemo. The main editor shows the code for RBACExample.java, which includes a static class Environment and a checkAccess method. The console at the bottom displays the output of the application, showing access control results for various users and actions.

```

shirley1 - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer
  > ABACDemojava
  > CloudSimCustomScheduler
  > CloudSimLogForensics
  > CloudSimResourceManagement
  > CloudSimSecureFileSharing
  > DataAnonymization
  > ImageEncryption
    > JRE System Library [JavaSE-1.8]
    > src
      > (default package)
        > ImageEncryptionAES.java
  > ImageObfuscation
    > JRE System Library [JavaSE-1.8]
    > src
      > (default package)
        > ImageObfuscationDemo.java
  > RBACDemo
    > JRE System Library [JavaSE-1.8]
    > src
      > (default package)
        > RBACExample.java

PriorityData... CustomSched... ResourceMana... LogForensic... ABACDemoexam...
25 static class Environment {
26     LocalDateTime accessTime;
27
28     Environment(LocalDateTime accessTime) {
29         this.accessTime = accessTime;
30     }
31 }
32
33 static boolean checkAccess(User user, Resource resource, Environment env) {
34
35     LocalDateTime start = LocalDateTime.of(9, 0);
36     LocalDateTime end = LocalDateTime.of(17, 0);
37
38     if (env.accessTime.isBefore(start) || env.accessTime.isAfter(end)) {
39         return false;
40     }
41
42     if ((user.role.equals("Doctor") || user.role.equals("Nurse")) &&
43         user.department.equals("Cardiology")) {
44         return true;
45     }
46
47     return false;

```

```

Problems @ Javadoc Declaration Console X Eclipse IDE for Java Developers 2026-03 M3
<terminated> RBACExample [Java Application] C:\Program Files\Java\jdk-11.0.30\bin\javaw.exe (22-Feb-2026, 10:48:41 pm - 10:48:41 pm)
----- ROLE BASED ACCESS CONTROL -----
Alice (ADMIN) -> Access GRANTED for upload
Alice (ADMIN) -> Access GRANTED for download
Alice (ADMIN) -> Access GRANTED for delete
Bob (USER) -> Access GRANTED for upload
Bob (USER) -> Access GRANTED for download
Bob (USER) -> Access DENIED for delete
Charlie (GUEST) -> Access GRANTED for view
Charlie (GUEST) -> Access DENIED for upload

```

RESULT:

Thus, the Role-Based Access Control (RBAC) mechanism was successfully implemented. Users were granted or denied access strictly based on their assigned roles.

Exp No:9	IMPLEMENT AN ATTRIBUTE-BASED ACCESS CONTROL MECHANISM BASED ON A PARTICULAR SCENARIO
Date:	

AIM:

To implement an Attribute-Based Access Control (ABAC) mechanism using Java based on user, resource, and environmental attributes.

PROCEDURE:

STEP 1: Open Eclipse IDE

STEP 2: Create a New Java Project

- Click File → New → Java Project
- Project Name: ABACDemo
- Click Finish

STEP 3: Create a New Java Class

- Right-click on src
- Click New → Class
- Class Name: ABACExample
- Tick public static void main(String[] args)
- Click Finish

PROGRAM CODE:

```
import java.time.LocalDateTime;

public class ABACExample {

    // User class

    static class User {

        String name;

        String role;

        String department;

        User(String name, String role, String department) {

            this.name = name;

            this.role = role;

            this.department = department;

        }

    }

}
```

```

    }
}

// Resource class
static class Resource {
    String recordType;

    Resource(String recordType) {
        this.recordType = recordType;
    }
}

// Environment class
static class Environment {
    LocalTime accessTime;

    Environment(LocalTime accessTime) {
        this.accessTime = accessTime;
    }
}

// ABAC Policy Check
static boolean checkAccess(User user, Resource resource, Environment env) {
    LocalTime start = LocalTime.of(9, 0);
    LocalTime end = LocalTime.of(17, 0);

    // Time-based rule
    if (env.accessTime.isBefore(start) || env.accessTime.isAfter(end)) {
        return false;
    }
}

```

```

// Role and Department rule
if ((user.role.equals("Doctor") || user.role.equals("Nurse")) &&
    user.department.equals("Cardiology")) {
    return true;
}
return false;
}

public static void main(String[] args) {
    User doctor = new User("Dr. Alice", "Doctor", "Cardiology");
    User nurse = new User("Nina", "Nurse", "Cardiology");
    User outsider = new User("Bob", "Doctor", "Orthopedics");

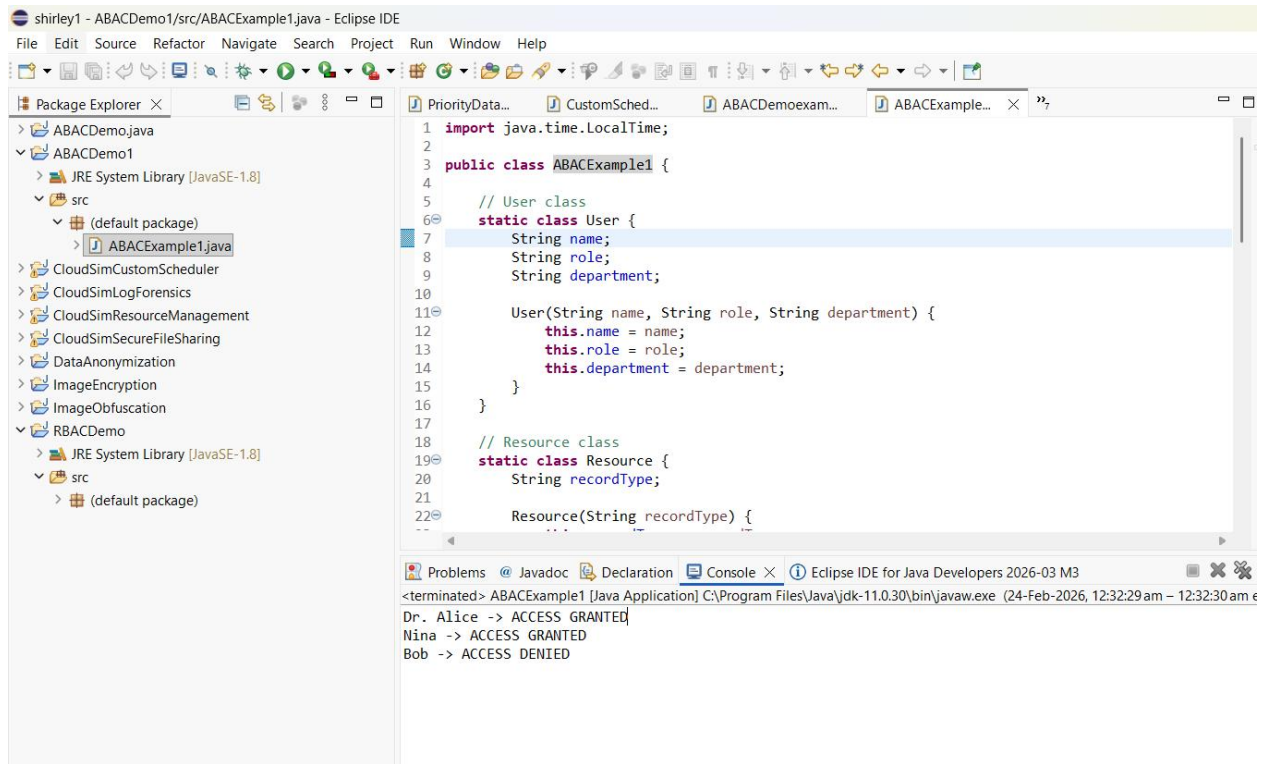
    Resource record = new Resource("Heart Patient Record");
    Environment env = new Environment(LocalTime.of(10, 30));
    testAccess(doctor, record, env);
    testAccess(nurse, record, env);
    testAccess(outsider, record, env);
}

static void testAccess(User user, Resource resource, Environment env) {
    if (checkAccess(user, resource, env)) {
        System.out.println(user.name + " -> ACCESS GRANTED");
    } else {
        System.out.println(user.name + " -> ACCESS DENIED");
    }
}
}

```

}

OUTPUT:



The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure, including the ABACDemo1 package and its sub-packages. The main editor window shows the ABACExample1.java file, which contains the following code:

```
1 import java.time.LocalDateTime;
2
3 public class ABACExample1 {
4
5     // User class
6     static class User {
7         String name;
8         String role;
9         String department;
10
11         User(String name, String role, String department) {
12             this.name = name;
13             this.role = role;
14             this.department = department;
15         }
16     }
17
18     // Resource class
19     static class Resource {
20         String recordType;
21
22         Resource(String recordType) {
```

The console output at the bottom shows the results of the program execution:

```
<terminated> ABACExample1 [Java Application] C:\Program Files\Java\jdk-11.0.30\bin\javaw.exe (24-Feb-2026, 12:32:29 am - 12:32:30 am)
Dr. Alice -> ACCESS GRANTED
Nina -> ACCESS GRANTED
Bob -> ACCESS DENIED
```

RESULT:

Thus, the Attribute-Based Access Control (ABAC) mechanism was successfully implemented and access decisions were made based on user, resource, and environmental attributes.

Exp No:10	DEVELOP A LOG MONITORING SYSTEM WITH INCIDENT MANAGEMENT IN THE CLOUD
Date:	

AIM:

To develop a Cloud Log Monitoring System with Incident Management that detects abnormal log entries and generates incidents automatically.

PROCEDURE:

STEP 1: Open Eclipse

STEP 2: Create Java Project

Project Name: CloudLogMonitoring

STEP 3: Create Java Class

Class Name: LogMonitoringSystem

STEP 5: Save the Program

STEP 6: Run the Program

PROGRAM CODE:

```
import java.util.*;
```

```
public class LogMonitoringSystem {
```

```
    static int failedLoginCount = 0;
```

```
    static List<Incident> incidentList = new ArrayList<>();
```

```
    static Scanner sc = new Scanner(System.in);
```

```
    // ===== INCIDENT CLASS =====
```

```
    static class Incident {
```

```
        String message;
```

```
        String severity;
```

```
        Date timestamp;
```

```
        Incident(String message, String severity) {
```

```
            this.message = message;
```

```

        this.severity = severity;

        this.timestamp = new Date();
    }

    void displayIncident() {
        System.out.println("-----");
        System.out.println("Incident ID : " + timestamp.getTime());
        System.out.println("Severity   : " + severity);
        System.out.println("Message   : " + message);
        System.out.println("Time      : " + timestamp);
        System.out.println("-----");
    }
}

// ===== LOG MONITORING FUNCTION =====

static void monitorLog(String logEntry) {
    System.out.println("Log Received: " + logEntry);
    if (logEntry.contains("FAILED_LOGIN")) {
        failedLoginCount++;
        if (failedLoginCount >= 3) {
            Incident incident = new Incident(
                "Multiple Failed Login Attempts Detected",
                "HIGH");
            incident.displayIncident();
            incidentList.add(incident);
            failedLoginCount = 0;
        }
    }
}

```



```

    }
}
else if (logEntry.contains("CRITICAL")) {
    Incident incident = new Incident(
        "Critical System Failure Detected",
        "CRITICAL");
    incident.displayIncident();
    incidentList.add(incident);
}
else {
    System.out.println("Log recorded successfully.");
}
}

// ===== VIEW INCIDENT HISTORY =====

static void viewIncidents() {
    if (incidentList.isEmpty()) {
        System.out.println("No incidents recorded.");
    } else {
        System.out.println("\n===== INCIDENT HISTORY =====");
        for (Incident incident : incidentList) {
            incident.displayIncident();
        }
    }
}

// ===== MAIN METHOD =====

```

```
public static void main(String[] args) {  
    int choice = 0;  
    do {  
        System.out.println("\n===== CLOUD LOG MONITORING MENU =====");  
        System.out.println("1. Enter Log");  
        System.out.println("2. View Incident History");  
        System.out.println("3. Exit");  
        System.out.print("Enter your choice: ");  
        if (sc.hasNextInt()) {  
            choice = sc.nextInt();  
            sc.nextLine(); // clear buffer  
            switch (choice) {  
                case 1:  
                    System.out.print("Enter Log Message: ");  
                    String logInput = sc.nextLine();  
                    monitorLog(logInput);  
                    break;  
                case 2:  
                    viewIncidents();  
                    break;  
                case 3:  
                    System.out.println("Exiting System...");  
                    break;  
                default:  
                    System.out.println("Invalid choice! Please enter 1, 2 or 3.");  
            }  
        }  
    } while (choice != 3);  
}
```

```

        }

    } else {

        System.out.println("Invalid input! Please enter a number.");

        sc.next(); // clear wrong input

    }

} while (choice != 3);

sc.close();

}

}

```

OUTPUT:

===== CLOUD LOG MONITORING MENU =====

1. Enter Log
2. View Incident History
3. Exit

Enter your choice: 1

Enter Log Message: FAILED_LOGIN

Log Received: FAILED_LOGIN

Log recorded successfully.

Enter your choice: 1

Enter Log Message: FAILED_LOGIN

Enter your choice: 1

Enter Log Message: FAILED_LOGIN

Incident ID : 1708720000000

Severity : HIGH

Message : Multiple Failed Login Attempts Detected

Time : Tue Feb 24 02:10:00 IST 2026

RESULT:

Thus the system successfully detected suspicious activities and generated incidents with appropriate severity levels.